

《基于 .NET 架构的 QoS 感知 Web 服务动态调用平台的设计与实现》

毕业论文答辩问题与参考答案

软件工程专业 • 20 个高概率问题

问题目录

1. 为什么选择这个课题？它解决了什么问题？
2. 论文中的“动态调用”具体动态在哪里？
3. 为什么选择五个 QoS 指标？权重如何确定？
4. QoS 指标如何归一化？为什么采用这种方法？
5. 如果两个服务得分相同，系统如何处理？
6. 为什么采用分层架构？各层职责是什么？
7. 为什么同时实现 REST 和 SOAP/WSDL？
8. SOAP 动态调用为何不用生成的代理类？
9. 多服务组合如何实现并行？如何证明不是串行？
10. 超时、重试、故障转移和熔断有什么区别？
11. 为什么连续失败两次后熔断二十秒？
12. 健康检查和熔断是否重复？
13. 系统如何保证并发条件下的数据安全？
14. 为什么使用 JSON 而不是数据库保存历史？
15. 系统测试如何设计？测试结果说明了什么？
16. Docker 在本项目中解决了什么问题？
17. 项目有哪些安全问题？如何改进？
18. 当前系统的主要不足是什么？
19. 如果扩展到大量服务或生产环境，如何改造？
20. 本人的主要工作和项目创新点是什么？

一、选题与算法

问题1：为什么选择这个课题？它解决了什么实际问题？

参考回答：选择该课题是因为在分布式系统中，同一种业务能力往往存在多个可替换的服务提供者，它们功能相近，但响应时间、成功率、成本和负载不同。如果调用地址被固定在代码中，一旦服务变慢或故障，系统只能等待超时或人工修改配置。我的项目通过服务注册、QoS 评价、动态选择和故障转移，使调用方能够在运行时选择当前更合适的服务。

这个课题既包含软件工程中的需求分析、架构设计、模块解耦和测试，也涉及分布式系统中的服务发现、质量评价与容错。它不是单纯实现几个 API，而是建立“注册—评价—选择—调用—反馈”的闭环。项目最终支持四类业务、十个候选服务、REST 和 SOAP 两种协议，并能通过 Docker 一键启动，具有完整的工程交付形态。

30 秒版本：固定调用无法适应服务质量变化。本项目用 QoS 把多个候选服务量化排名，在运行时选择最优端点，并在失败时自动切换，从而提高可用性和可解释性。

问题2：论文中的“动态调用”具体动态在哪里？只是动态 URL 吗？

参考回答：动态调用不仅是更换 URL，它至少包含四层动态性。第一，候选端点来自服务注册表，业务编排器不直接依赖具体 Provider；第二，系统在每次请求时根据最新 QoS 快照和用户权重重新排名；第三，统一调用器根据 ServiceEndpoint 的 Protocol 字段在 REST 和 SOAP 路径之间选择；第四，SOAP 路径在运行时下载并校验 WSDL，根据注册的 SOAP Action 构造 XML，而不是依赖编译期代理。

因此，动态对象包括“调用谁、按什么质量偏好选择、使用什么协议、失败后切换给谁”。当前注册表是内存实现，但上层依赖 IServiceRegistry 接口，将来可以替换为数据库或注册中心。动态不是无约束地执行任意代码，而是在明确契约和安全边界内进行运行时绑定。

问题3：为什么选择可用性、成功率、响应时间、成本和负载五个 QoS 指标？权重如何确定？

参考回答：这五项分别覆盖可靠性、实际调用结果、性能、经济性和资源状态，能够形成较完整且易解释的评价。可用性反映服务是否在线；成功率反映调用质量；响应时间代表用户等待；成本模拟商业调用代价；负载反映当前资源繁忙程度。指标数量过少会遗漏重要决策因素，数量过多则不利于演示和权重解释。

默认权重为 0.35、0.25、0.20、0.10、0.10，主要考虑演示系统首先保证可用和成功，再考虑速度，最后考虑成本和负载。这些数值不是宣称适用于所有业务，而是一套可配置基线。用户可以在页面或 API 中调整权重。例如实时查询可以提高响应时间权重，批量任务可以提高成本权重。系统会校验权重均在零到一之间且总和为一。

答辩注意：不要回答“权重是随便设的”。应说明它是基于业务优先级设置的默认策略，并强调系统支持配置和替换算法。

问题4：QoS 指标如何归一化？为什么采用最小—最大归一化？

参考回答：各指标量纲不同，响应时间以毫秒计、成本以金额计、成功率是比例，不能直接相加。对于可用性和成功率这类效益型指标，使用 $(x - \min) / (\max - \min)$ ；对于响应时间、成本和负载这类成本型指标，使用 $1 - (x - \min) / (\max - \min)$ 。结果都落在零到一之间，再乘以权重求和。

选择该方法是因为实现简单、计算复杂度低、结果直观，适合候选数量较少且需要解释的本科项目。当同一指标的最大值等于最小值时，所有候选没有差异，统一归一化为一，避免除零。它的不足是对极端值敏感，而且分数依赖当前候选集合。生产环境可以改用固定区间、Z-score、TOPSIS 或基于历史分布的方法。

问题5：如果两个服务综合得分相同，系统如何处理？

参考回答：系统先按综合分数降序排序，得分相同时按端点名称进行确定性的升序排序。这样相同输入和相同 QoS 数据会得到稳定结果，方便自动化测试和问题复现，而不会因为字典遍历顺序产生随机选择。

更高级的策略可以采用轮询、带权随机、最少连接数或基于近期调用次数的负载均衡。当前选择稳定排序是为了突出 QoS 决策并保持可解释性。如果实际生产环境担心同分端点流量集中，可以在同分组内再使用轮询作为二级策略。

二、架构与协议

问题6：为什么采用分层架构？各层的职责是什么？

参考回答：分层的目的不是增加项目数量，而是隔离变化。Domain 层定义端点、QoS、请求和结果等稳定概念；Application 层定义接口并实现评分与组合编排；Infrastructure 层处理 HTTP、SOAP、注册表、指标、熔断、健康检查和文件存储；Web 层负责 Razor 页面和 API；MockProviders 提供可控的模拟服务。

依赖方向由外向内，上层业务不依赖具体网络或存储。例如 CompositeOrchestrator 只依赖 IServiceInvoker 和 IServiceRegistry，在单元测试中可以使用伪实现，不需要启动服务器。未来把 JSON 历史替换为数据库，或者把内存注册表替换为服务注册中心，都不必修改评分算法。这体现了单一职责和依赖倒置原则。

问题7：为什么同时实现 REST 和 SOAP/WSDL？

参考回答：REST 是当前常见的轻量接口方式，SOAP/WSDL 则更贴合课题最初的 Web Service 动态调用背景。只做 REST 容易变成普通的 HTTP 聚合项目，只做 SOAP 又无法体现现代 .NET 服务开发。因此系统让两种协议共同进入同一个注册、QoS 排名、编排和故障转移流程。

这也验证了架构的协议扩展能力：上层只识别 IServiceInvoker，端点通过 Protocol 指明 REST 或 SOAP。测试时可以调整成本和负载权重，让“经典 SOAP 天气”在三个天气候选中排名第一，从而明确证明 SOAP 不是孤立演示，而是真正参与动态选择。

问题8：SOAP 动态调用为什么不用 Visual Studio 生成的代理类？

参考回答：生成代理类会在编译期把 WSDL 契约转成固定类型，开发方便，但端点和操作形态基本已经静态绑定，不能充分体现运行时动态发现。本项目首次调用时获取 WSDL，解析并确认 QueryWeather 操作存在，再根据注册表中的地址和 SOAP Action 构造 SOAP 1.1 Envelope，发送 text/xml 请求并解析结果。

这种方法的优势是过程透明、没有生成代码依赖，并且 WSDL 地址可以在运行时变化。局限是目前只支持一个简单操作，没有完整实现复杂 XSD、数组、继承、WS-Security 等。如果要做通用 SOAP 平台，应使用成熟的元数据导入和动态代理框架，或者基于 CoreWCF 扩展。当前实现针对论文目标进行了合理裁剪。

问题9：多服务组合如何实现并行？如何证明不是串行？

参考回答：编排器把每个 `ServiceTask` 映射为 `ExecuteOneAsync` 任务，然后使用 `Task.WhenAll` 一次等待全部任务。每个类别独立进行候选获取、评分和网络调用，因此天气、物流、汇率和通知可以同时等待 I/O。某一类别内部的候选切换仍然是顺序的，因为只有前一候选失败后才需要下一候选。

证明方式一是检查代码结构和调用时间线；方式二是比较耗时。容器端到端测试中四类调用总耗时约三百毫秒，接近最慢单项，而不是四项延迟之和。如果串行执行，80、160、220 等模拟延迟会累加，总时间明显更长。`Task.WhenAll` 不会自动创建四个线程，它利用异步 I/O 在等待期间释放线程，这是需要区分的概念。

三、可靠性与数据

问题10：超时、重试、故障转移和熔断有什么区别？项目实现了哪些？

参考回答：超时限制一次调用最长等待时间；重试是对同一个端点再次发送请求；故障转移是放弃当前端点并调用另一个候选；熔断是在持续失败后暂时禁止访问某端点。它们解决的问题不同，不能混为一谈。

本项目实现两秒 HttpClient 超时、按 QoS 排名的候选故障转移和连续失败熔断，没有对同一端点进行自动重复重试。原因是模拟操作虽然基本幂等，但通用通知或支付类操作重复执行可能产生副作用；直接切换候选更能展示服务选择。如果未来增加重试，需要限制次数、加入退避和抖动，并只对明确可重试的异常与幂等操作启用。

问题11：为什么连续失败两次后熔断二十秒？这个参数有理论依据吗？

参考回答：两次和二十秒是面向演示的策略参数，不是普适理论最优值。阈值为一会因偶发网络抖动过早熔断，阈值过大又难以在答辩中观察；二次失败能体现“连续故障”判断。二十秒足够观察熔断状态，又不会让演示等待太久。

生产环境应根据调用频率、错误预算、服务恢复时间和历史数据配置，通常还会使用滚动时间窗口、失败比例、最小吞吐量与半开探测。项目中的价值在于熔断行为和接口已经建立，具体阈值可以配置化演进。答辩时应主动说明参数是工程权衡，不要把演示参数包装成理论定值。

问题12：健康检查和熔断是否功能重复？

参考回答：不重复。健康检查是后台主动探测，主要判断 Provider 宿主是否在线；熔断依据真实业务调用失败，被动识别某个具体端点是否持续异常。一个宿主的 /health 可能正常，但某个业务路由持续返回错误，这时健康检查认为在线，熔断器仍会隔离故障端点。反过来，如果整个宿主关闭，健康检查可以在用户请求之前就把全部端点过滤掉。

项目中健康监控每十秒按基地址分组检查，熔断状态按 EndpointId 单独维护。二者在编排器获取候选后依次过滤，形成宿主级和端点级两道保护。

问题13：系统如何保证并发条件下的数据安全？

参考回答：QoS 指标、健康状态、熔断状态和 WSDL 缓存都可能被并行任务访问，因此使用 ConcurrentDictionary。计数和平均耗时通过 AddOrUpdate 原子更新。JSON 历史存储把加载、插入、裁剪和写文件放在同一个 lock 临界区内，避免两个请求交叉覆盖文件。

这种保证只覆盖单进程实例。若 Web 横向扩展为多个容器，各实例的内存状态彼此独立，文件锁也不能跨容器协调。生产多实例方案需要把指标和熔断状态放入 Redis，把历史放入数据库，并使用事务或分布式锁。这是当前方案明确的扩展边界。

问题14：为什么使用 JSON 而不是数据库保存历史？

参考回答：当前历史只保存最近一百次调用摘要，字段少、查询简单，JSON 可以直接查看，不增加数据库安装、迁移和连接配置，符合演示项目的最小复杂度原则。Docker 通过命名卷把 App_Data 持久化，容器重建后历史仍保留。

JSON 的不足是查询能力弱、并发写入受限、无法支持多实例，也不适合大量数据。若需求变成长期统计、条件查询或多用户并发，应使用 SQLite 或 PostgreSQL，并通过 EF Core 管理模型与迁移。因此选 JSON 是针对当前规模的工程权衡，而不是认为文件存储优于数据库。

四、测试、部署与扩展

问题15：系统测试如何设计？测试结果说明了什么？

参考回答：测试分三层。单元测试验证评分器在速度优先和成本优先时的选择、权重校验、第一候选失败后的切换以及熔断器状态；集成测试验证四类注册候选数量和类别映射；端到端测试启动真实进程或 Docker 容器，访问健康接口、WSDL 和组合 API。

最终解决方案编译为零错误、零警告，十个自动化测试全部通过。容器中两个服务保持 Up，健康接口返回 healthy，WSDL 包含 QueryWeather。调整权重后，QoS 引擎选择 SOAP 天气候选并成功返回 SOAP 1.1 结果。这说明不仅单个算法正确，从入口、容器网络、WSDL 发现到 Provider 响应的完整链路也已验证。

答辩注意：测试通过不能证明软件没有缺陷。应补充尚未进行正式压力测试、长稳测试和安全渗透测试。

问题16：Docker 在本项目中解决了什么问题？不用 Docker 是否也能运行？

参考回答：不用 Docker 也能运行，两个项目可以通过 dotnet run 分别启动。Docker 的价值是把 .NET Runtime、发布结果、端口、容器网络、环境变量和历史卷固化，使展示环境可重复，避免在另一台机器上重新配置多进程地址。Web 容器通过服务名 providers 访问 Provider，不依赖宿主 localhost。

Compose 同时定义两个容器的构建和启动顺序，并配置 restart 策略。多阶段 Dockerfile 用 SDK 镜像编译、Runtime 镜像运行，减小最终镜像。Docker 不是业务算法的一部分，而是软件工程交付和可移植性的体现。

问题17：项目有哪些安全问题？如何改进？

参考回答：当前定位是本机演示，没有身份认证和权限控制，故障注入 API 对任何能够访问端口的人开放；服务使用 HTTP 而非 HTTPS；SOAP 客户端允许按配置访问 WSDL，如果配置来源不可信可能带来服务器端请求伪造；JSON 文件也没有加密。这些在生产环境都必须改进。

改进措施包括接入 ASP.NET Core Identity 或外部身份提供商，为管理接口设置角色授权；启用 HTTPS 和证书；对白名单域名、协议和端口进行校验；限制请求体与 XML 大小；对凭据使用 Secret Manager 或容器秘密；增加速率限制、审计日志和依赖漏洞扫描。系统当前通过不保存完整业务载荷、使用 XDocument 自动转义和仅映射本机演示端口降低了部分风险，但不能替代正式安全设计。

问题18：当前系统的主要不足是什么？

参考回答：第一，服务注册信息和部分 QoS 基准仍是静态或模拟数据；第二，成功率和平均响应时间采用累计方式，对近期变化不够敏感；第三，JSON 历史不适合多实例和复杂查询；第四，SOAP 只实现简单操作，没有覆盖复杂 Schema 和 WS-Security；第五，熔断器是简化状态机；第六，尚未进行正式性能压力、安全和长时间稳定性测试。

这些不足与项目目标有关。本课题重点验证 QoS 动态选择、协议无关调用和完整工程流程，因此优先采用透明、可演示的实现。论文中明确写出边界，并给出滑动窗口、数据库、OpenTelemetry、成熟弹性库和 Kubernetes 等演进方向。

问题19：如果扩展到大量服务或生产环境，如何改造？

参考回答：首先将内存注册表替换为配置中心或服务注册中心，并为端点元数据建立管理 API。其次把 QoS 指标和熔断状态放入 Redis 或专用指标系统，使用滑动窗口计算近期质量；历史进入关系数据库或时序数据库。再次将各候选拆成独立服务，在 Kubernetes 中使用健康探针、自动扩缩容和服务发现。

调用层应采用成熟的 resilience pipeline，按操作幂等性配置重试、超时、熔断和并发限制；通过 OpenTelemetry 传播关联 ID，统一收集日志、指标与追踪。评分层可以引入按业务命名的策略、约束过滤、TOPSIS 或机器学习。安全方面必须加入认证授权、HTTPS、密钥管理和网络策略。现有接口分层使这些替换可以逐步完成，而不必推翻业务模型。

问题20：本人的主要工作是什么？项目有哪些创新点？

参考回答：主要工作包括需求分析、QoS 指标与评分模型设计、分层架构和领域模型设计、十个模拟候选实现、REST 与 SOAP 动态调用、并行编排、健康检查、熔断和历史存储、Razor 页面、自动化测试、Docker 部署以及论文和 UML 文档。

创新点应实事求是地描述。第一，把传统 SOAP/WSDL 和现代 REST 候选放入同一套 QoS 选择与故障转移流程，而不是分别做两个孤立示例；第二，SOAP 客户端运行时解析 WSDL，不使用静态代理，更符合动态调用主题；第三，选择过程保留分项得分、排名和尝试链路，具有可解释性；第四，项目从算法、代码、测试到容器交付形成完整闭环。它不是提出全新的学术算法，而是在软件工程层面对多个技术点进行了系统集成和可验证实现。

收尾表达：我的核心贡献不是发明一种新的 QoS 理论，而是把服务注册、质量评价、双协议动态调用、弹性控制、可视化、测试和部署组织成一个真正可运行、可解释、可复现的系统。

■ 答辩表达建议

回答时先用一句话直接给结论，再补充设计依据和项目证据，最后主动说明边界。遇到未实现功能不要回避，可以回答“当前目标范围内采用了某方案，生产化时将如何改造”。尽量引用项目中的具体数字，例如四类服务、十个候选、五项指标、两秒超时、十秒健康检查、连续两次失败、二十秒熔断、最近一百条历史、两个容器和十个自动化测试。具体数字能够证明回答来自实际实现。